

Website themes

A theme controls how your website looks: layout, navigation, colors, and typography. Every *Calepin* site uses one. The default theme works with zero configuration, and when you want more control, you can copy any built-in theme into your project and edit it like the rest of your source files.

Built-in themes

Calepin ships with two themes:

- **calepin** (the default): a documentation site layout with sidebar navigation, a top bar with your logo and a GitHub link, previous and next page links, a table of contents, a dark mode toggle, copy buttons on code blocks, and a view switcher that shows each page as rendered HTML, Typst source, or PDF. This documentation site uses it.
- **academic**: a personal homepage layout with top navigation instead of a sidebar, designed for profile pages, teaching materials, and publication lists. Run `calepin new academic` to scaffold a complete starter site built on it.

Choosing a theme

Set theme in your site's `calepin.toml`. It accepts three kinds of values:

```
theme = "academic"      # a built-in theme: calepin or academic
theme = "themes/my-theme" # a theme directory in your project
theme = false           # no theme: raw, unstyled output
```

The same values work with `--theme` on `calepin compile` and `calepin watch`, and inside a document with `#calepin.setup(theme: ...)`. When several are set, the command line wins, then the document, then `calepin.toml`.

Customizing a theme

Built-in themes are compiled into the *Calepin* binary, so you cannot edit them directly. Instead, copy one into your project and edit the copy:

```
calepin new theme          # copies the default theme to themes/calepin/
calepin new theme --theme academic # copies the academic theme to themes/academic/
```

Then point your site at the copy:

```
theme = "themes/calepin"
```

The copy is yours: edit its HTML, CSS, JavaScript, and Typst files freely, and check them into version control. *Calepin* upgrades will never touch them.

Two tips:

- Keep only the files you actually change. Anything you delete from your copy falls back to the built-in default, so a theme directory containing a single CSS file is perfectly valid, and the remaining files document exactly what you customized.
- Never edit files under `.calepin/`. That directory is regenerated on every build, and changes there are silently lost.

Inside a theme

A theme is just a directory. *Calepin* looks for three entry files, one per kind of output:

```

themes/my-theme/
  site.html      # layout for website pages
  document.html  # layout for a single document rendered to HTML
  paged.typ      # Typst styling for PDF, SVG, and PNG output
  partials/      # reusable template fragments
  styles/        # CSS files, loaded in filename order
  scripts/       # JavaScript files, loaded in filename order

```

Every file is optional. When an entry file is missing, that kind of output uses the default *calepin* theme instead, including the default's own partials, styles, and scripts. The built-in *academic* theme works this way: it only provides `site.html`, and falls back to the default for single-document HTML and paged output.

One special case: an empty `paged.typ` turns paged styling off entirely, while a missing one inherits the default styling.

Writing templates

`site.html` and `document.html` use the MiniJinja template language, which follows the familiar Jinja2 syntax. Files in `partials/` can be included with `{% include "partials/name.html" %}`.

On website builds, templates receive a `site` object describing the site and the current page:

- `site.title`, `site.description`, `site.base_url`: site identity from `calepin.toml`
- `site.logo`, `site.logo_alt`, `site.home_url`, `site.github_url`: branding and top-bar links
- `site.sidebar`, `site.sidebar_sections`: sidebar navigation entries
- `site.navbar_left`, `site.navbar_center`, `site.navbar_right`: navbar entries by region
- `site.toc`: table of contents for the current page
- `site.page_title`, `site.current_url`: current page metadata
- `site.language`, `site.languages`, `site.translations`: multilingual site data

Each entry in `site.sidebar`, `site.sidebar_sections`, and the navbar regions exposes `href`, `label`, `label_html`, `active`, and `widget`. Plain links have `widget = none`. Widget entries carry whatever string was configured: the built-in themes recognize `widget = "theme"` (dark mode toggle) and `widget = "language"` (language picker), and a custom theme can invent its own names, such as `search`, and render them by checking `item.widget`.

For the variables available when rendering a single document, see *HTML themes*.

Reusable snippets

You do not have to write all the CSS and JavaScript yourself. *Calepin* exposes its own building blocks to every theme through the `snippets` object:

```

<style>{{ snippets.css.theme }}</style>
<style>{{ snippets.css.code }}</style>
<style>{{ snippets.css.widgets }}</style>
<script>{{ snippets.js.copy_code }}</script>
<script>{{ snippets.js.language_picker }}</script>
<script>{{ snippets.js.theme_toggle }}</script>

```

- `snippets.css.theme` is the shared visual base of the built-in themes: typography, heading scale, colors, and code, output, and figure defaults.
- `snippets.css.code` styles code blocks and computed output.

- `snippets.css.widgets`, together with `snippets.js.theme_toggle` and `snippets.js.language_picker`, powers the dark mode toggle (elements marked `data-calepin-theme-toggle`) and the language picker (selects marked `data-calepin-language-picker`).
- `snippets.js.copy_code` adds copy buttons to code blocks.

Prefer these snippets over copying CSS or JavaScript out of the built-in themes: your theme stays small, and it inherits fixes and improvements from future *Calepin* releases.

Styling PDF output

`paged.typ` is a plain Typst file, not a template. When rendering PDF, SVG, or PNG output, *Calepin* inserts it ahead of each page's source, after its own code chunk rules, so your set and show rules apply to the whole document.

Calepin also stages reusable Typst snippets under `/.calepin/snippets/typst/`. The default theme's `paged.typ` imports `code-block.typ` from there to style source blocks. A minimal `paged.typ` that does the same:

```
#import "/.calepin/snippets/typst/code-block.typ": code-block

#show raw.where(block: true): it => {
  if sys.inputs.at("calepin-target", default: "paged") == "html" {
    it
  } else {
    code-block(it)
  }
}
```

Source and PDF views

The default theme's view switcher shows each page as rendered HTML, Typst source, or PDF.

The source view reads a JSON script embedded in each generated HTML page, so it always works. The PDF view links to the matching `.pdf` file, which only exists when PDF rendering is enabled. If you build with `--format html`, no PDF files are generated, and the PDF view has nothing to show.